

Trabajo de Aprendizaje por Refuerzo: Caminante Bípido

Jorge Bengoa Pinedo

Bogurad Barański Barańska

Máster TECI - Redes neuronales y aprendizaje estadístico

28 de febrero de 2026

1. Introducción

En este trabajo se busca abordar mediante aprendizaje por refuerzo el problema del caminante bípido (*Bipedal Walker*) que consiste en enseñar a un robot en el entorno virtual *Gymnasium* a andar. Para ello, se dispone de los datos de 24 sensores, tal y como se puede ver en la figura 1, que conforman el espacio de observaciones. A partir de esta información, el robot debe tomar decisiones dinámicas mediante sus 4 actuadores, situados cada uno en una articulación. Estos actuadores se encargan de estabilizar y ejercer movimiento sobre el robot aplicando una velocidad de rotación comprendida entre -1 y 1 rad/s .

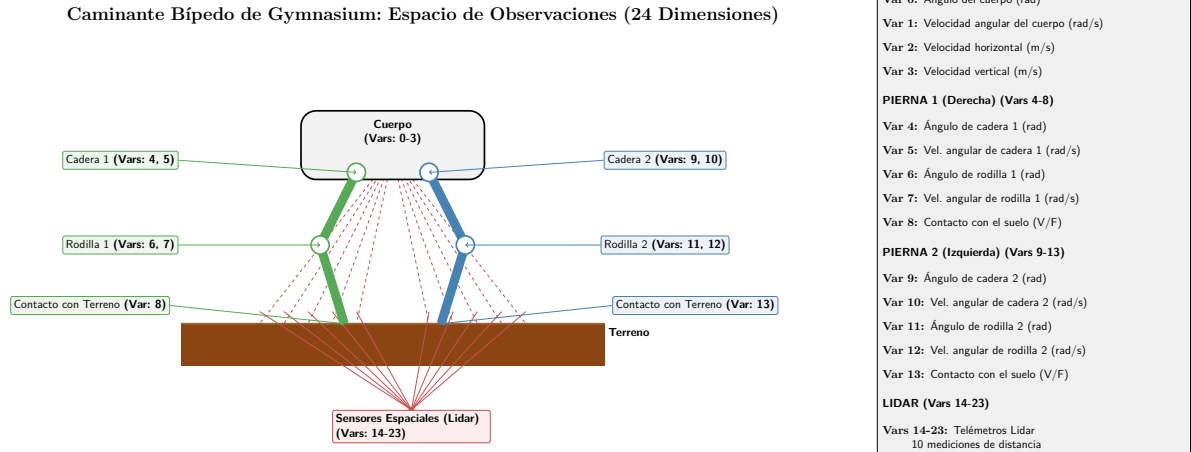


Figura 1: Mapping of the 24-Dimensional Observation Space for the BipedalWalker-v3 Environment

Por tanto, en este trabajo, para la resolución del problema planteado anteriormente se propone abordarlo mediante aprendizaje por refuerzo y, en concreto, utilizando el paradigma de *Soft Actor Critic* descrito en los fundamentos matemáticos a continuación. Este se implementa en *Python* mediante librerías estándar de *stable_baselines3*, donde primero se realiza una búsqueda exhaustiva de hiperparámetros mediante optimización con *optuna*, para luego entrenar un modelo con los mejores parámetros y ver si el agente aprende a andar.

2. Fundamentos matemáticos

3. Procesos de decisión de Markov

La siguiente sección se ha escrito de acuerdo a [1].

Los procesos de decisión de Markov (MDP) sirven para modelar el aprendizaje mediante interacción para conseguir un objetivo. Los MDP comprenden dos elementos:

- Agente: es el que aprende y toma decisiones.
- Entorno: comprende todo lo que es externo al agente e interactúa con el agente.

Cuando el agente toma una decisión, el entorno presenta una nueva situación al agente. Además, le entrega una recompensa. Mediante sus acciones, el agente busca maximizar la recompensa obtenida a lo largo del tiempo. La figura 2 refleja estas interacciones.

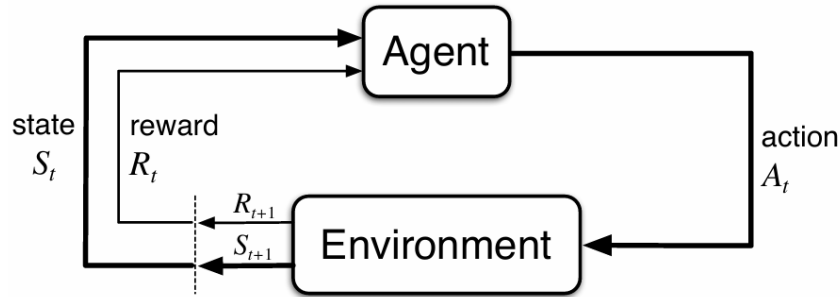


Figura 2: Interacción entre agente y entorno a través de un proceso de decisión de Markov.

Sean $\mathcal{S}$, $\mathcal{A}$ y $\mathcal{R}$ los conjuntos de estados, acciones y recompensas, respectivamente, entonces:

$$\begin{aligned}
 p &: \mathcal{S} \times \mathcal{R} \times \mathcal{S} \times \mathcal{A} \rightarrow [0, 1] \\
 p(s', r \mid s, a) &= \Pr S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a \\
 \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) &= 1, \quad \forall s \in \mathcal{S}, a \in \mathcal{A}(s)
 \end{aligned} \tag{1}$$

Donde $s, s' \in \mathcal{S}$, $r \in \mathcal{R}$, $a \in \mathcal{A}$. A partir de la ecuación anterior, se puede calcular la probabilidad marginal del siguiente estado:

$$\begin{aligned}
 p &: \mathcal{S} \times \mathcal{S} \times \mathcal{A} \rightarrow [0, 1] \\
 p(s' \mid s, a) &= \Pr\{S_t = s' \mid S_{t-1} = s, A_{t-1} = a\} = \sum_{r \in \mathcal{R}} p(s', r \mid s, a)
 \end{aligned} \tag{2}$$

Y la recompensa esperada:

$$\begin{aligned}
 r &: \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R} \\
 r(s, a) &= \mathbb{E}[R_t \mid S_{t-1} = s, A_{t-1} = a] = \sum_{r \in \mathcal{R}} r \cdot p(r \mid s, a) = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(s', r \mid s, a)
 \end{aligned} \tag{3}$$

3.1. Aprendizajes episódico y continuo

El objetivo del agente es maximizar la recompensa esperada en el largo plazo. Esto implica conocer cómo una acción presente repercute en los sucesivos pasos temporales. Según se tenga un estado final o no, se distingue entre:

- Aprendizaje episódico: cada episodio tiene una duración T finita, que no tiene por qué ser fija (p.ej. partidas de ajedrez). La recompensa esperada a maximizar se podría escribir como:

$$G_t = R_{t+1} + R_{t+2} + \dots + R_T \quad (4)$$

- Aprendizaje continuo: las interacciones agente-entorno no tienen un fin o este es muy lejano (p.ej. un robot *pick and place*). En tal caso, la definición anterior podría tomar el valor ∞ . Para evitarlo, se define la recompensa descontada:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} = R_{t+1} + \gamma \sum_{k=1}^{\infty} \gamma^{k-1} R_{t+k+1} = R_{t+1} + \gamma G_{t+1} \quad (5)$$

Donde γ es el factor de descuento. Si $\gamma < 1$, el sumatorio converge a un valor finito. Cuanto mayor sea el valor de γ , mayor visión a futuro tendrá el agente.

Se puede adoptar una notación común a ambos problemas:

$$G_t = \sum_{k=t+1}^T \gamma^{k-t-1} R_k \quad (6)$$

Donde las condiciones $\gamma = 1$ y $T = \infty$ son mutuamente excluyentes y representan los casos episódico y continuo de forma excluyente.

3.2. Políticas y funciones de valor

Una política define la probabilidad de que el agente tome una determinada acción dado un estado. Se denotará por $\pi(a | s)$.

Por otro lado, las funciones de valor pueden ser:

- Funciones de estado-valor: determinan cómo de bueno es que el agente esté en un determinado estado:

$$v_{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s] = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right], \forall s \in \mathcal{S} \quad (7)$$

La función estado-valor óptima es:

$$v_*(s) = \max_{\pi} v_{\pi}(s), \forall s \in \mathcal{S} \quad (8)$$

- Funciones de acción-valor: determinan cómo de bueno es que el agente tome una determinada acción dado un estado:

$$q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a] = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right] \quad (9)$$

Se dice que una política es óptima si $v_*(s) \geq v_\pi(s), \forall s \in S, \forall \pi$. La función acción-valor óptima es:

$$q_*(s, a) = \max_{\pi} q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a], \forall s \in \mathcal{S}, a \in \mathcal{A} \quad (10)$$

A partir de las expresiones anteriores se deduce la ecuación de expectativa de Bellman sobre la función estado-valor:

$$\begin{aligned} v_\pi(s) &= \mathbb{E}_\pi[G_t \mid S_t = s] = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\ &= \sum_{a \in \mathcal{A}} \pi(a \mid s) \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a] && \text{(Ley de la probabilidad total)} \\ &= \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) \cdot \\ &\quad \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a, S_{t+1} = s', R_{t+1} = r] && \text{(Ley de la probabilidad total)} \\ &= \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_{t+1} = s'] && \text{(Propiedad Markoviana)} \\ &= \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) \mathbb{E}_\pi[r + \gamma G_{t+1} \mid S_{t+1} = s'] && \text{(Propiedad Markoviana)} \\ &= \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) [r + \gamma \mathbb{E}_\pi[G_{t+1} \mid S_{t+1} = s']] \\ &= \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_\pi(s')] && (11) \end{aligned}$$

Análogamente, se puede obtener la ecuación de optimalidad de Bellman sobre la función estado-valor (nótese que la función estado-valor óptima corresponde con seguir la política óptima partiendo de la acción que maximiza la recompensa):

$$\begin{aligned} v_*(s) &= \max_{a \in \mathcal{A}(s)} q_{\pi^*}(s, a) \\ &= \max_a \mathbb{E}_{\pi^*}[G_t \mid S_t = s, A_t = a] \\ &= \max_a \mathbb{E}_{\pi^*}[R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a] \\ &= \max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a] \\ &= \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_*(s')] && (12) \end{aligned}$$

Las mismas ecuaciones de Bellman se pueden deducir para la función acción-valor:

$$q_\pi(s, a) = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a] = \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_\pi(s')] \quad (13)$$

$$q_*(s, a) = \mathbb{E}_\pi[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \mid S_t = s, A_t = a] = \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma \max_{a'} q_*(s', a') \right] \quad (14)$$

Conocido el valor de $v_*(s)$, bastaría con mirar el siguiente estado con ese valor esperado y seleccionar la acción que conduce a dicho estado. Como la información a largo plazo está codificada en la propia función estado-valor óptima, moverse de forma codiciosa (*greedy*), es decir,

mirando solo al siguiente instante, ya implica tener en cuenta el futuro. Conocido $q_*(s, a)$, el problema es aún más fácil ya que no haría falta conocer que acción conduce al siguiente estado. Tan solo habría que seleccionar la acción cuyo valor sea óptimo.

Lógicamente, resolver las ecuaciones de Bellman puede no ser factible debido a que se necesitaría memoria y potencia computacional infinitas. Por lo tanto, se usan aproximaciones de las funciones estado-valor y acción-valor. Aunque los resultados son subóptimos, suelen funcionar razonablemente bien. Las políticas óptimas resultantes tienden a poner más esfuerzo en aprender de los estados que ocurren frecuente y menos esfuerzo en los menos habituales.

4. Aprendizaje por diferencias temporales

Idealmente, la política óptima se deriva de las ecuaciones de optimalidad de Bellman. Sin embargo, su resolución no es trivial, por lo que aparecen distintos esquemas para aproximar $v_*(s)$ y $q_*(s, a)$:

- Métodos de Monte-Carlo: no requieren un modelo del entorno. Las funciones de valor aproximadas se actualizan al final de cada episodio, estimando la esperanza a partir de una muestra.
- Programación dinámica: no requiere esperar al final de cada episodio para su actualización (permiten aprendizaje continuo), ya que calcula la esperanza exacta de las funciones de valor al suponer conocido un modelo del entorno.
- Diferencias temporales: aproximan las funciones valor sin necesidad de esperar el final de cada episodio ni de un modelo del entorno.

El enfoque de diferencias temporales combina las ventajas de los métodos de Monte-Carlo y la programación dinámica, haciéndolo flexible ante diversas situaciones.

4.1. Predicción TD

El problema de predicción consiste en estimar la función de valor v_π a partir de la experiencia siguiendo una política π . El método TD(0) es el más simple de todos. Actualiza su estimación en el instante $t + 1$ utilizando la recompensa inmediata observada y la estimación de v_π en el siguiente estado:

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)] = V(S_t) + \alpha \delta_t \quad (15)$$

Donde $\alpha \in (0, 1]$ es la tasa de aprendizaje y $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ es el error TD. Este último mide la diferencia entre la estimación actual, $V(S_t)$, y una estimación mejorada respaldada por la experiencia, $R_{t+1} + \gamma V(S_{t+1})$.

Otra ventaja de los métodos TD frente a Monte-Carlo es que son más robustos ante acciones arbitrarias de exploración. En los métodos de Monte-Carlo, este tipo de acciones podría distorsionar el valor de la esperanza, por lo que habría que descartar la iteración. Los métodos TD, por contra, aprenden de cada transición.

Los métodos TD convergen a la media de v_π si se sigue una política fija, π , y α es constante y suficientemente pequeño. La probabilidad de convergencia es 1 si α se reduce gradualmente, cumpliendo ciertas condiciones. No obstante, muchas de las pruebas de convergencia solo aplican al caso tabular.

Respecto a la velocidad de convergencia, los métodos TD convergen más rápido que Monte-Carlo de forma práctica, aunque no haya una demostración matemática. Esto ocurre para el formato online y batch (acumulando el error TD sobre todas las muestras disponibles antes de actualizar).

4.2. Control TD

El problema de control consiste en estimar $q_\pi(s, a)$. Por lo tanto, la actualización depende de los pares estado-acción. Existen distintas variantes de los métodos TD para este tipo de problemas:

- SARSA: es el caso más simple:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)] \quad (16)$$

Es un algoritmo on-policy (la misma política seguida para la transición de estados es la que se actualiza) en el que continuamente se estima q_π a la vez que se va modificando la política π para que tienda a la codicia. Se empieza con una política que permite la exploración con una cierta probabilidad. Cada vez se favorece más las decisiones de explotación (seleccionar las recompensas que maximizan las acciones) en contra de dicha exploración.

- Q-learning: es un algoritmo off-policy que converge a q_* :

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)] \quad (17)$$

La política de comportamiento para moverse por los estados puede elegirse libremente, lo que favorece la exploración. A medida que se van observando los estados, la política objetivo aprende a seleccionar la acción que maximiza la recompensa. Una ventaja de los métodos off-policy es que no importa cómo se han generado las muestras a la hora de actualizar la política objetivo, ya que ambas están desagregadas. Los métodos on-policy se actualizan de acuerdo a los estados explorados según la política actual. Es decir, no pueden reutilizar datos pasados porque fueron generados con una política diferente.

- Expected SARSA:

$$\begin{aligned} Q(S_t, A_t) &\leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \mathbb{E}_\pi [Q(S_{t+1}, A_{t+1}) \mid S_{t+1}] - Q(S_t, A_t)] \\ &= Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \sum_a \pi(a \mid S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t) \right] \end{aligned} \quad (18)$$

En vez de evaluar el siguiente par estado-acción, usa su valor esperado. La política objetivo π puede ser la misma que la de comportamiento (on-policy) o distinta (off-policy). Reduce la varianza asociada a elegir una única acción A_{t+1} de forma aleatoria a cambio de un mayor coste computacional.

- Double-Q-learning: utiliza dos funciones de acción-valor para desacoplar la acción de la evaluación:

$$Q_1(S_t, A_t) \leftarrow Q_1(S_t, A_t) + \alpha [R_{t+1} + \gamma Q_2(S_{t+1}, \arg \max_a Q_1(S_{t+1}, a)) - Q_1(S_t, A_t)] \quad (19)$$

$$Q_2(S_t, A_t) \leftarrow Q_2(S_t, A_t) + \alpha [R_{t+1} + \gamma Q_1(S_{t+1}, \arg \max_a Q_2(S_{t+1}, a)) - Q_2(S_t, A_t)] \quad (20)$$

El Q-learning es optimista cuando las recompensas son ruidosas. Intuitivamente, se puede imaginar la situación en que los valores reales $q(s, a)$ son todos cero, pero sus estimaciones $Q(s, a)$ presentan incertidumbre. El máximo de los valores reales es cero, mientras que el máximo estimado es positivo. Esto se conoce como sesgo de maximización.

El double-Q-learning aborda este problema calculando simultáneamente dos funciones acción-valor: $Q_1(s, a)$ y $Q_2(s, a)$. Pero en cada iteración se actualiza solamente una de ellas a partir de la evaluación según la otra. Si una recompensa es ruidosa en torno a cero, será raro que tome valores positivos constantemente. Por ejemplo, para la primera muestra se actualiza Q_1 , que ve una recompensa muy alta. Si se usara Q-learning, otros pares estado-acción aumentarían su valor solamente por la expectativa de volver a repetir la jugada. Sin embargo, Q_2 evalúa el par estado-acción sin haber visto tal recompensa. Su percepción será que dicho par estado-acción no tiene tanto valor. Este proceso se repite actualizando arbitrariamente cada función.

Referencias

- [1] A. G. Barto, “Reinforcement learning: An introduction. by richard’s sutton,” *SIAM Rev*, vol. 6, no. 2, p. 423, 2021.